

CC, SI & IA – 2ª série

Automatização de Testes de Software

2025

Bibliografia Básica

- PRESSMAN, Roger S; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.
<https://integrada.minhabiblioteca.com.br/#/books/9786558040118>
- SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

Aula 10 – Teoria

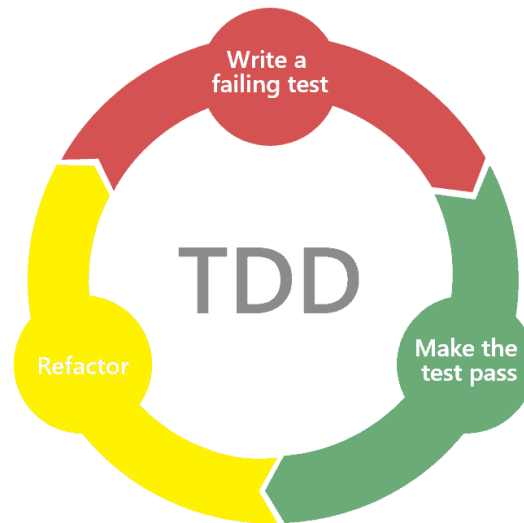
Bibliografia Complementar

- CALVETTI, Robson. Testes de Software. Material de aula, São Paulo, 2022.

O que é?

» **TDD – Test-Driven Development:**

- Também conhecido por Desenvolvimento Orientado a Testes, é uma prática de desenvolvimento de *software* que se concentra na criação de testes de unidade antes de escrever o código de produção.



O que é?

» **TDD – Test-Driven Development:**

Segue um ciclo de desenvolvimento iterativo e incremental que consiste em três etapas principais:

1. Vermelho – RED: FALHA!

- Nesta primeira etapa, o desenvolvedor escreve um teste de unidade que descreve uma pequena parte da funcionalidade que deseja implementar;
- Esse teste ainda não deve passar, pois o código de produção correspondente ainda não foi escrito;
- O objetivo é criar um teste que falhe, indicando que a funcionalidade desejada ainda não está implementada.

O que é?

» **TDD – Test-Driven Development:**

Segue um ciclo de desenvolvimento iterativo e incremental que consiste em três etapas principais:

2. Verde – GREEN: **SUCESSO!**

- Na segunda etapa, o desenvolvedor escreve o código de produção necessário para fazer com que o teste de unidade criado na etapa anterior passe com sucesso;
- O objetivo é implementar a funcionalidade de maneira suficientemente simples e não definitiva para que o teste passe.

O que é?

» **TDD – Test-Driven Development:**

Segue um ciclo de desenvolvimento iterativo e incremental que consiste em três etapas principais:

3. Amarelo – YELLOW: REFATORAÇÃO!

- Na terceira etapa, após o teste passar com sucesso, o desenvolvedor pode refatorar o código, melhorando sua estrutura e qualidade, sem alterar seu comportamento observado externamente;
- O objetivo é manter o código limpo e organizado.

Quais são?

» **Benefícios do TDD – *Test-Driven Development*:**

- Melhoria da Qualidade:

Ajuda a garantir que o código seja testado exaustivamente desde o início, o que pode levar a um *software* mais confiável e com menos erros;

- Projeto Incremental:

O desenvolvimento é orientado por testes, o que significa que só se escreve código para implementar as funcionalidades necessárias, o que evita a criação de código desnecessário;

Quais são?

» **Benefícios do TDD – *Test-Driven Development*:**

- Documentação Viva:

Os testes de unidade servem como documentação viva do comportamento esperado do código. Isso facilita a compreensão do código por outros desenvolvedores;

- Facilita a Refatoração:

Como os testes são executados automaticamente, a refatoração do código é mais segura, pois qualquer regressão, como uma introdução de erro, é rapidamente detectada pelos testes.

Quais são?

» **Benefícios do TDD – *Test-Driven Development*:**

- Diminuição de Erros/Defeitos/Falhas:

Reduções significativas nas taxas de defeitos encontrados nas fases de testes unitários e de integração;

- Redução de Esforços Finais dos Projetos:

Reduções significativas do esforço nas fases finais dos projetos, ao custo de um aumento moderado no esforço nas fases iniciais do desenvolvimento;

Quais são?

» **Benefícios do TDD – *Test-Driven Development*:**

- Aumento da Qualidade do Código:

Através do seu uso, o código-fonte atinge graus mais altos de qualidade técnica;

- Aumento da Manutenibilidade:

Facilita a manutenção do *software*;

- Aumento da Modularidade:

Torna o projeto de software mais modular.

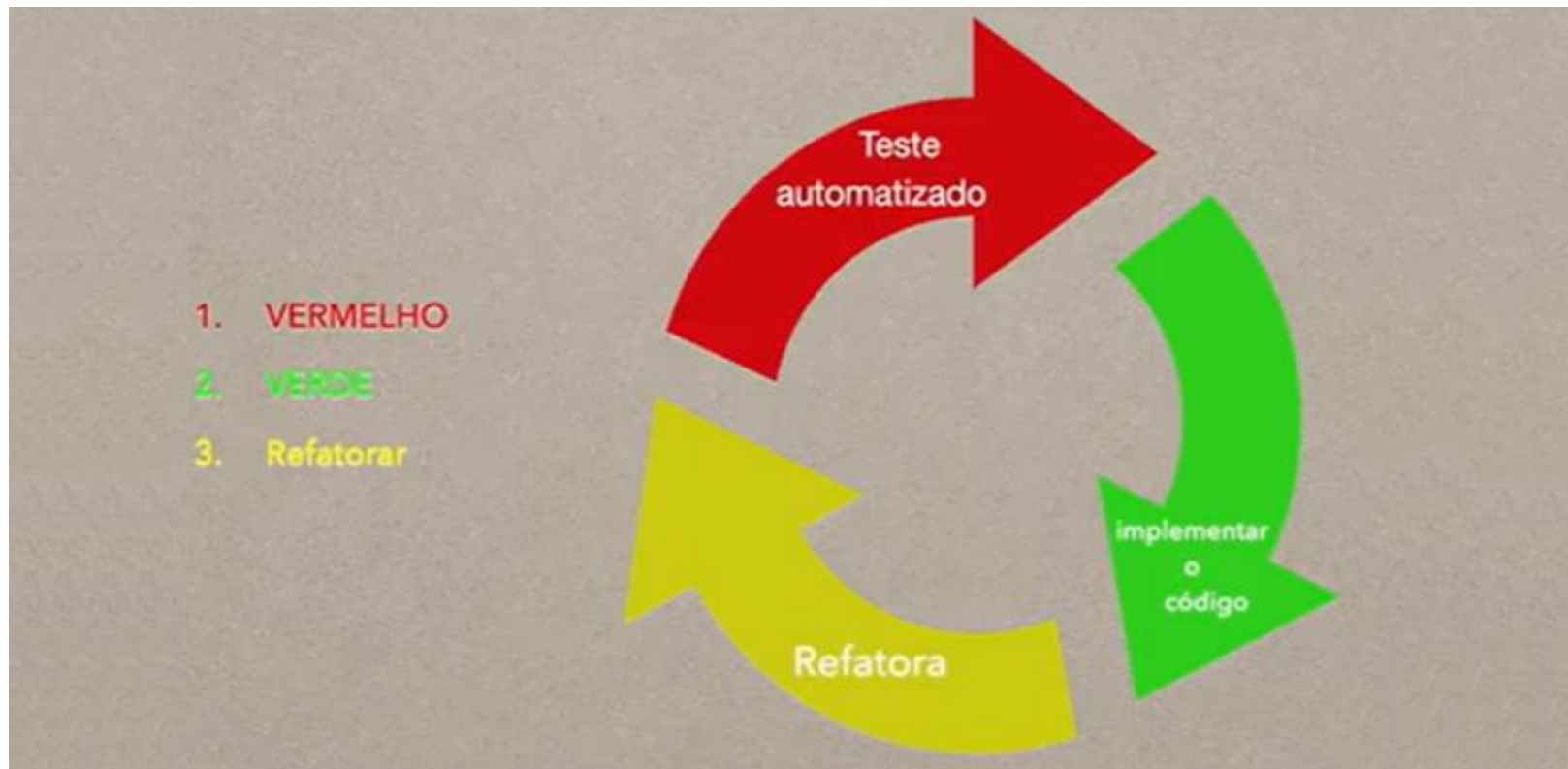
Como é?

» Ciclo do TDD – *Test-Driven Development*:

- O **TDD** define duas regras, apenas, para serem seguidas, obrigatoriamente, que também determinam a ordem das tarefas da programação:
 1. Escrever um código novo apenas se o teste automatizado falhar;
 2. Eliminar a duplicação de código.
- Daí surge o conhecido “mantra” do **TDD**:
 1. Vermelho;
 2. Verde; e
 3. Refatorar.

Como é?

» Ciclo do TDD – *Test-Driven Development*:



Como é?

» Ciclo do TDD – *Test-Driven Development*:

1. Escrever um teste para uma determinada funcionalidade, sem escrever o código dessa funcionalidade (*test first*);
2. Executar esse teste, sem ainda ter implementado essa funcionalidade, visando que o teste indique falha;
3. Implementar essa funcionalidade, de forma simples, rápida e provisória (*baby Steps*), visando ser aprovada em seu teste;
4. Executar o teste, novamente, para a funcionalidade alterada, visando que, agora, o teste indique sucesso;
5. Refatorar o código da funcionalidade para que seja aprovada, agora com seu código confiável, limpo e definitivo;

Como é?

» **Ciclo do TDD – *Test-Driven Development*:**

6. Executar o teste, novamente, para a funcionalidade refatorada, visando que o teste indique sucesso, novamente;
7. Escolher uma nova funcionalidade e reiniciar o ciclo do desenvolvimento guiado por testes;
8. A medida que se for avançando nos testes e nas funcionalidades aprovadas, todos os testes feitos anteriormente devem ser executados novamente, repetindo-se esse ciclo, aumentando, assim, a confiança no código que está sendo produzido.

Aula 10 – Teoria

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

1. Definir o ambiente para executar o método de ordenação e seus respectivos testes: a classe **ExemploTDD**:

```
1 public class ExemploTDD
2 {
3     public static void main(String args[])
4     {
5         OrdenaTest tdd = new OrdenaTest();
6     }
7 }
8
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

2. Criar a classe de testes e os respectivos casos de testes para o requisito solicitado: a classe **OrdenaTest**:

```
1 public class OrdenaTest
2 { public OrdenaTest()
3   { int proposto[]    = new int[] {10, 9};
4     int esperado[]    = new int[] {9, 10};
5     int inesperado[]  = new int[] {9};
6
7     Ordena teste = new Ordena();
8     teste.ordenaNumerosCrescentes(proposto);
9
10    System.out.println("Teste de Ordenação\n=====");
11    System.out.println("Ficou com o mesmo tamanho: " + caso1Test(proposto.length, inesperado.length));
12    System.out.println("Ordenou com sucesso.....: " + caso2Test(proposto, esperado));
13  }
14 }
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

3. Criar os métodos que executam os respectivos casos de testes para o requisito solicitado: a classe **OrdenaTest**:

```
15     public boolean caso1Test(int tamprop, int tamesp)
16     {   boolean resp = true;
17         if(tamprop != tamesp)    resp = false;
18         return    resp;
19     }
20
21     public boolean caso2Test(int prop[], int esp[])
22     {   return    numerosIguais(prop, esp);
23     }
24
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

4. Criar os métodos auxiliares dos casos de testes para o requisito solicitado: a classe **OrdenaTest**:

```
25     public boolean numerosIguais(int nums1[], int nums2[])
26     {   boolean resultado = true;
27         for(int i = 0, j = 0; i < nums1.length; i++, j++)
28         {   if(nums1[i] != nums2[j])
29             {   resultado = false;
30                 i = nums1.length;
31             }
32         }
33         return resultado;
34     }
35 }
```

Aula 10 – Teoria

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

5. Iniciar o ciclo do TDD, escrevendo um código rápido para que a funcionalidade falhe nos testes já criados (**VERMELHO**): a classe **Ordena**:

```
1 public class Ordena
2 {   public void ordenaNumerosCrescentes(int vetor[])
3     {   // Não há código, portanto, não irá ordenar o vetor
4     }
5 }
6
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

6. Executar os casos de testes anteriormente criados para testar a funcionalidade recém escrita, objetivando se obter as falhas nos testes (**VERMELHO**): a classe **ExemploTDD**:

```
1 public class ExemploTDD
2 {
3     public static void main(String args[])
4     {
5         OrdenaTest tdd = new OrdenaTest();
6     }
7 }
8
```



```
---- exec: java ExemploTDD
Teste de Ordenação
=====
Ficou com o mesmo tamanho: false
Ordenou com sucesso.....: false

---- operation complete.
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

7. Avance o ciclo do TDD, escrevendo um código rápido para que a funcionalidade seja aprovada nos testes já criados (**VERDE**): a classe **OrdenaTest**:

```
1 public class OrdenaTest
2 { public OrdenaTest()
3   { int proposto[] = new int[] {10, 9};
4     int esperado[] = new int[] {9, 10};
5     int inesperado[] = new int[] {9};
6
7     Ordena teste = new Ordena();
8     teste.ordenaNumerosCrescentes(proposto);
9
10    System.out.println("Teste de Ordenação\n=====");
11    System.out.println("Ficou com o mesmo tamanho: " + caso1Test(proposto.length, esperado.length));
12    System.out.println("Ordenou com sucesso.....: " + caso2Test(proposto, esperado));
13  }
14 }
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

8. Reescreva um código rápido para que a funcionalidade seja aprovada nos testes já criados (**VERDE**): a classe **Ordena**:

```
1 public class Ordena
2 {   public void ordenaNumerosCrescentes(int vetor[])
3     {   int rascunho;
4         if(vetor[0] > vetor[1])
5         {   rascunho = vetor[1];
6             vetor[1] = vetor[0];
7             vetor[0] = rascunho;
8         }
9     }
10 }
11
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

9. Executar os casos de testes anteriormente criados para testar a funcionalidade recém escrita, objetivando o sucesso nos testes (**VERDE**): a classe **ExemploTDD**:

```
1 public class ExemploTDD
2 {
3     public static void main(String args[])
4     {
5         OrdenaTest tdd = new OrdenaTest();
6     }
7 }
8
```



```
----exec: java ExemploTDD
Teste de Ordenação
=====
Ficou com o mesmo tamanho: true
Ordenou com sucesso.....: true

----operation complete.
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

10. Avance o ciclo do TDD, melhorando o código em pequenos avanços, ou *Baby Steps* (REFATORAR), repetindo o ciclo até chegar ao código final da funcionalidade: a classe **Ordena**:

```
1 public class Ordena
2 {   public void ordenaNumerosCrescentes(int iVet[])
3     {   int    iA, iB, iT;
4         for(iA = 1; iA < iVet.length; iA++)
5         {   for(iB = iVet.length-1; iB >= iA; iB--)
6             {   if(iVet[iB-1] > iVet[iB])
7                 {   iT = iVet[iB-1];
8                     iVet[iB-1] = iVet[iB];
9                     iVet[iB] = iT;
10                }
11            }
12        }
13    }
14 }
15
```

Qual é?

» Exemplo de TDD – *Test-Driven Development*:

Ordenação de N números inteiros em um vetor:

11. Executar os casos de testes anteriormente criados para testar a funcionalidade recém escrita, objetivando o sucesso nos testes (**VERDE**): a classe **ExemploTDD**:

```
1 public class ExemploTDD
2 {
3     public static void main(String args[])
4     {
5         OrdenaTest tdd = new OrdenaTest();
6     }
7 }
8
```



```
----exec: java ExemploTDD
Teste de Ordenação
=====
Ficou com o mesmo tamanho: true
Ordenou com sucesso.....: true

----operation complete.
```

Aula 10 – Teoria

Demonstrações!

» **TDD – Test-Driven Development:**



Exemplos!

» **Automatização de Testes de *Software*:**

- A seguir, serão apresentados 6 exemplos práticos e graduais para o contexto *Java*, *VsCode*, *JUnit* e *TDD*, sendo:
 - ✓ 3 Testes Unitários (simples, médio e complexo);
 - ✓ 3 Testes de Integração (simples, médio e complexo).
- Cada exemplo incluirá:
 - ✓ A classe Java sob teste;
 - ✓ A classe de teste JUnit.

Aula 10 – Teoria

Exemplos!

» **Ambiente:**

- Antes, porém, o projeto deve estar configurado com a estrutura típica:

```
src/  
├─ main/java/  
└─ test/java/
```

Aula 10 – Teoria

Exemplos!

» Ambiente:

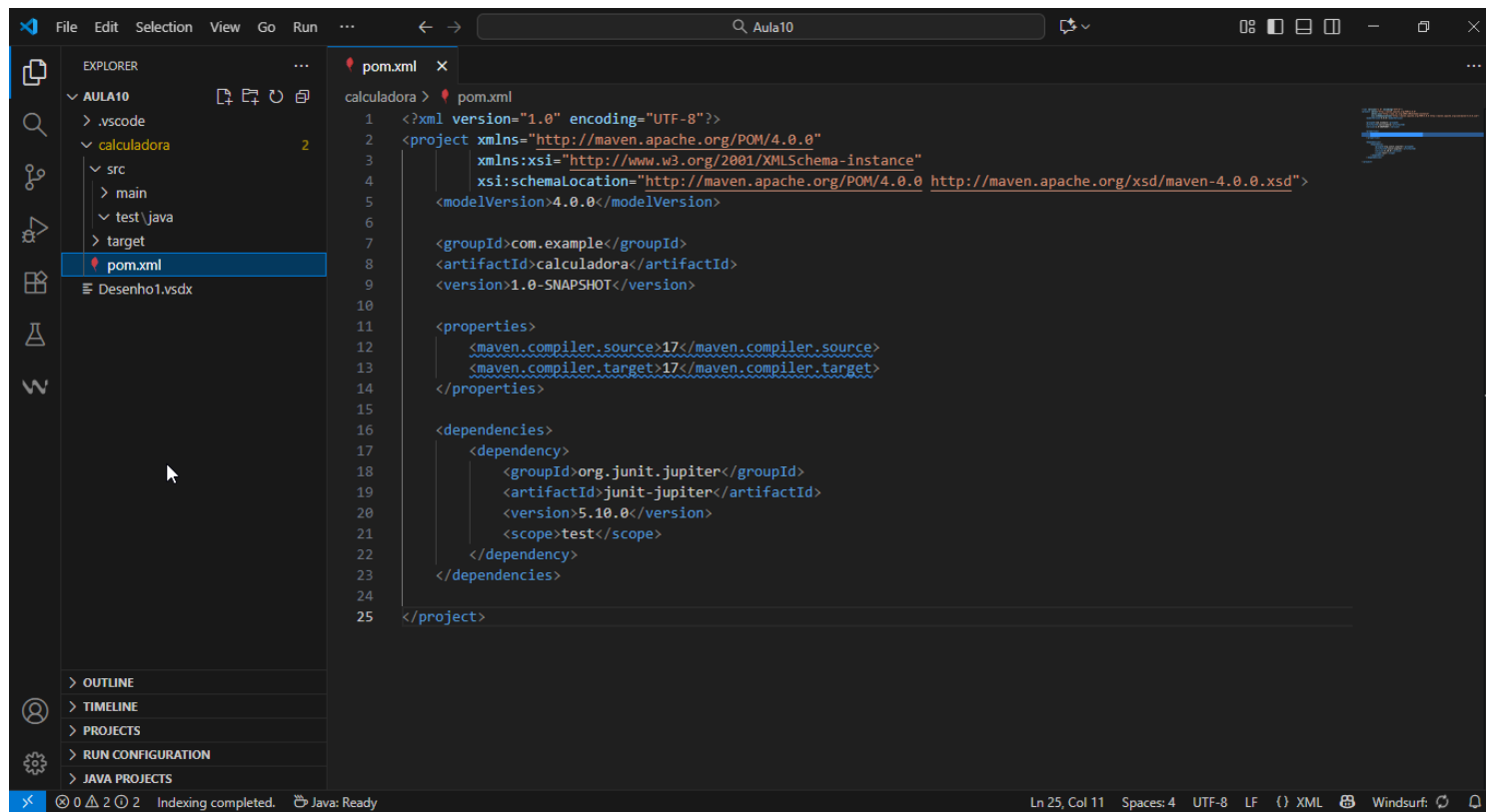
- E o *JUnit 5* incluído no *pom.xml* (para *Maven*):

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Exemplos!

» Ambiente:

- Arquivo *pom.xml* alterado:



The screenshot shows an IDE window titled 'Aula10'. The Explorer panel on the left shows a project structure with 'AULA10' as the root, containing '.vscode', 'calculadora', and 'Desenho1.vsd'. The 'calculadora' folder is expanded, showing 'src' (with 'main' and 'test\java' subfolders), 'target', and 'pom.xml'. The 'pom.xml' file is selected and its content is displayed in the main editor. The XML content is as follows:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.example</groupId>
8     <artifactId>calculadora</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>17</maven.compiler.source>
13         <maven.compiler.target>17</maven.compiler.target>
14     </properties>
15
16     <dependencies>
17         <dependency>
18             <groupId>org.junit.jupiter</groupId>
19             <artifactId>junit-jupiter</artifactId>
20             <version>5.10.0</version>
21             <scope>test</scope>
22         </dependency>
23     </dependencies>
24
25 </project>
```

The status bar at the bottom indicates 'Ln 25, Col 11', 'Spaces: 4', 'UTF-8', 'LF', 'XML', and 'Windsurf:'. The bottom left shows 'Indexing completed.' and 'Java: Ready'.

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora*

Calculadora

- somar(int, int): int

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest*
 - Funcionamento básico da função pura;
 - Uso de *assertEquals* e *assertNotEquals*.

CalculadoraTest

- testSomar(): void

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest.java*

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

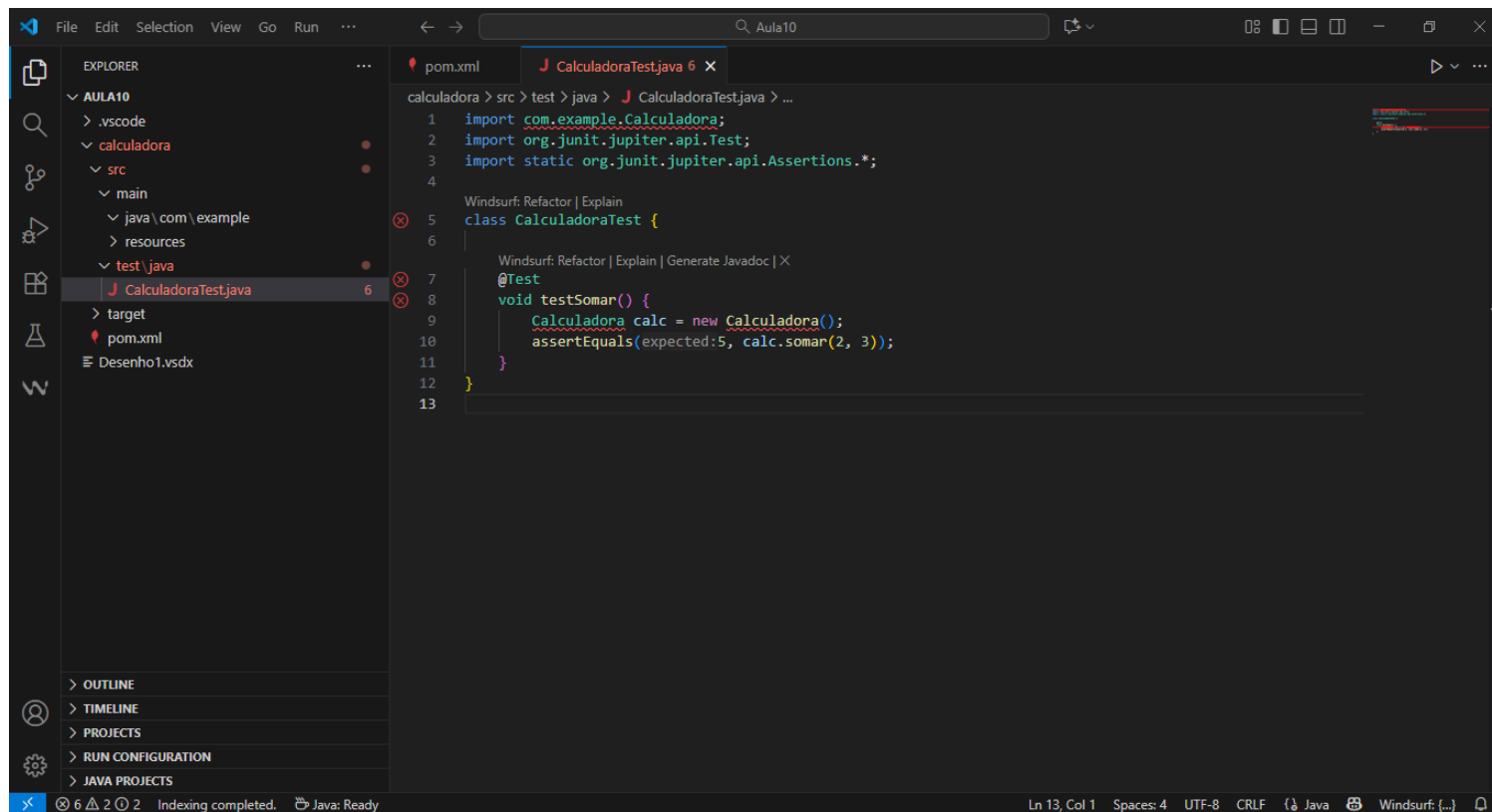
    @Test
    void testSomar() {
        Calculadora calc = new Calculadora();
        assertEquals(5, calc.somar(2, 3));
    }
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Classe de teste *CalculadoraTest.java* criada:



The screenshot shows an IDE window with the Explorer on the left and the Editor on the right. The Explorer shows a project structure with a package `test` containing a file `CalculadoraTest.java`. The Editor shows the code for `CalculadoraTest.java`, which includes imports for `com.example.Calculadora`, `org.junit.jupiter.api.Test`, and `org.junit.jupiter.api.Assertions.*`. The code defines a class `CalculadoraTest` with a test method `testSomar` that creates a `Calculadora` object and asserts the result of `somar(2, 3)` is 5. The status bar at the bottom indicates 'Ln 13, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java: Ready'.

```
1 import com.example.Calculadora;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class CalculadoraTest {
6
7     @Test
8     void testSomar() {
9         Calculadora calc = new Calculadora();
10        assertEquals(expected:5, calc.somar(2, 3));
11    }
12
13 }
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*
 - Fase 1 do TDD: **VERMELHO** → Reprovação Forçada!

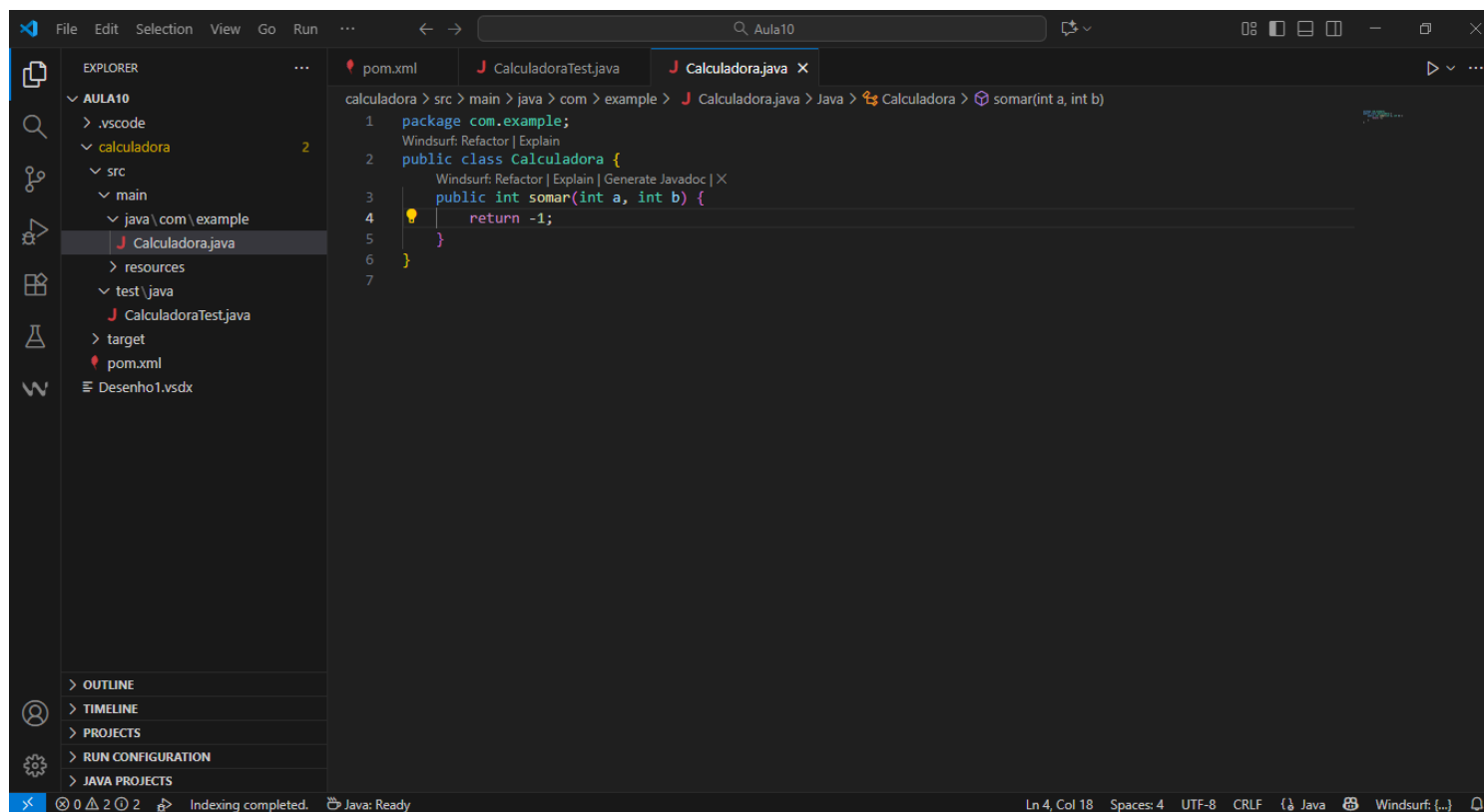
```
package com.exemplo;  
  
public class Calculadora {  
    public int somar(int a, int b) {  
        return -1;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the project structure for 'AULA10', including folders like 'src', 'main', and 'test', and files like 'Calculadora.java' and 'CalculadoraTest.java'. The main editor area shows the code for 'Calculadora.java', which defines a 'public class Calculadora' with a 'public int somar(int a, int b)' method. The code is as follows:

```
1 package com.example;
2 public class Calculadora {
3     public int somar(int a, int b) {
4         return -1;
5     }
6 }
```

The status bar at the bottom indicates 'Ln 4, Col 18', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java: Ready'.

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest.java*

The screenshot shows an IDE with the following components:

- EXPLORER:** Shows the project structure with folders for `AULA10`, `calculadora`, `src`, `main`, `resources`, `test\java`, and `target`. The file `CalculadoraTest.java` is selected under `test\java`.
- EDITOR:** Displays the code for `CalculadoraTest.java`. The code includes imports for `com.example.Calculadora`, `org.junit.jupiter.api.Test`, and `org.junit.jupiter.api.Assertions`. A test method `testSomar()` is shown, which creates a `Calculadora` instance and calls `somar(2, 3)`. A red error message indicates a test failure: `expected: <5> but was: <-1>org.opentest4j.AssertionFailedError at org.j...`.
- PROBLEMS:** Shows the test failure details.
- OUTPUT:** Displays the Maven command output, including the message: `Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test (default-cli) on project calculadora: There are test failures.`

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*
 - Fase 2 do TDD: VERDE → Aprovação Forçada!

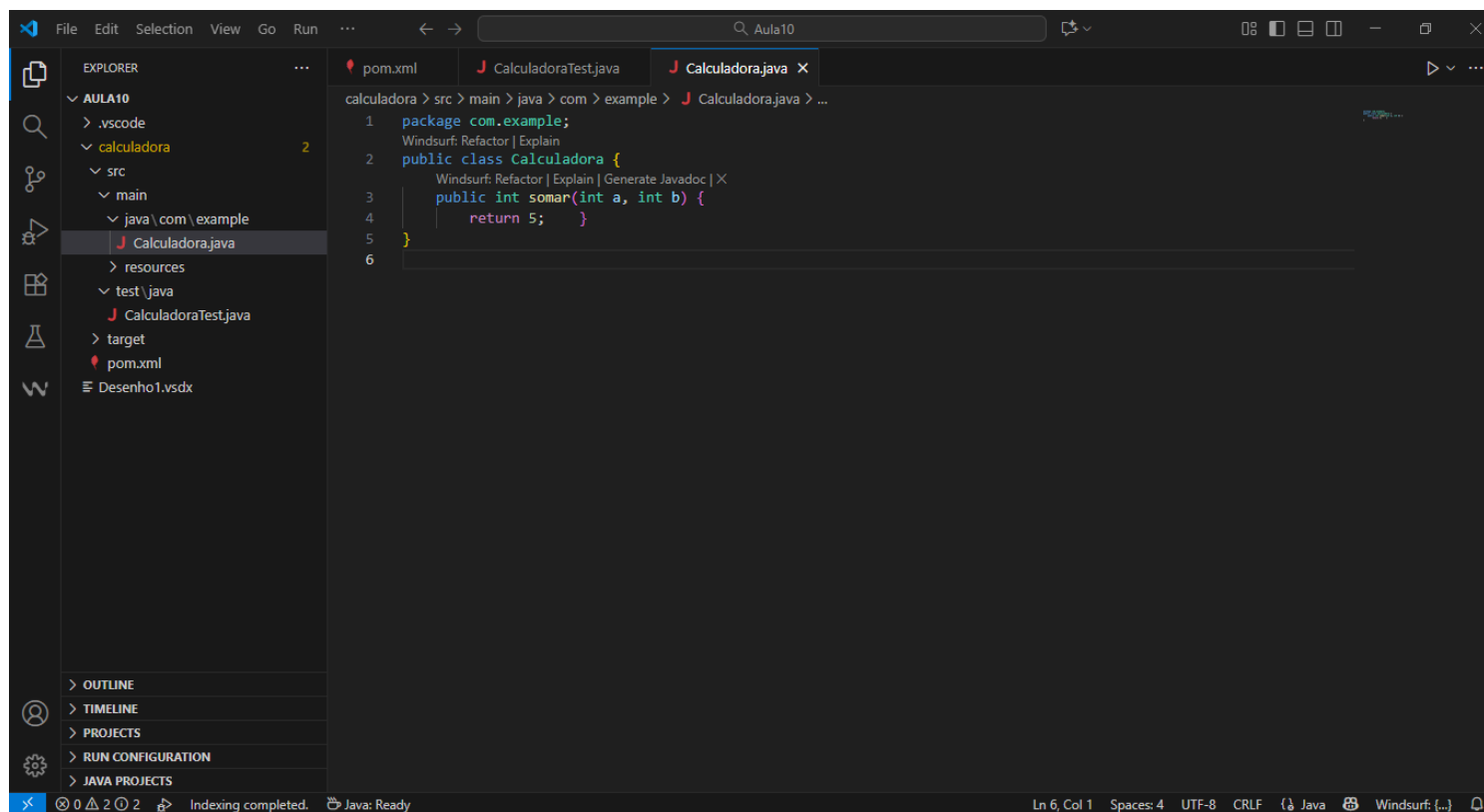
```
package com.exemplo;  
  
public class Calculadora {  
    public int somar(int a, int b) {  
        return 5;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a project structure with folders like .vscode, calculadora, src, main, java, and resources. The file Calculadora.java is selected. The main editor window shows the code for Calculadora.java, which includes a package declaration, a class declaration, and a static method named somar. The code is as follows:

```
1 package com.example;
2 public class Calculadora {
3     public int somar(int a, int b) {
4         return 5;
5     }
6 }
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest.java*

```
File Edit Selection View Go Run ...
AULA10
  .vscode
  calculadora
    src
      main
        java\com\example
          Calculadora.java
        resources
        test\java
          CalculadoraTest.java
        target
        pom.xml
Desenho1.vsd

pom.xml CalculadoraTest.java Calculadora.java
calculadora > src > test > java > J CalculadoraTest.java > ...
1 import com.example.Calculadora;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class CalculadoraTest {
6
7     @Test
8     void testSomar() {
9         Calculadora calc = new Calculadora();
10        assertEquals(expected:5, calc.somar(a:2, b:3));
11    }
12
13
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
Filter (e.g. text, !exclude, \esca... Java Single Test Run
Running CalculadoraTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in CalculadoraTest

Results:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Total time: 2.006 s
Finished at: 2025-10-11T23:05:00-03:00

Please start a debug session to evaluate expressions
Ln 13, Col 1 Spaces: 4 UTF-8 CRLF Java Windsurf: [...]
```

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*
 - Fase 3 do TDD: **REFATORAR** → Bucar Aprovação Real!

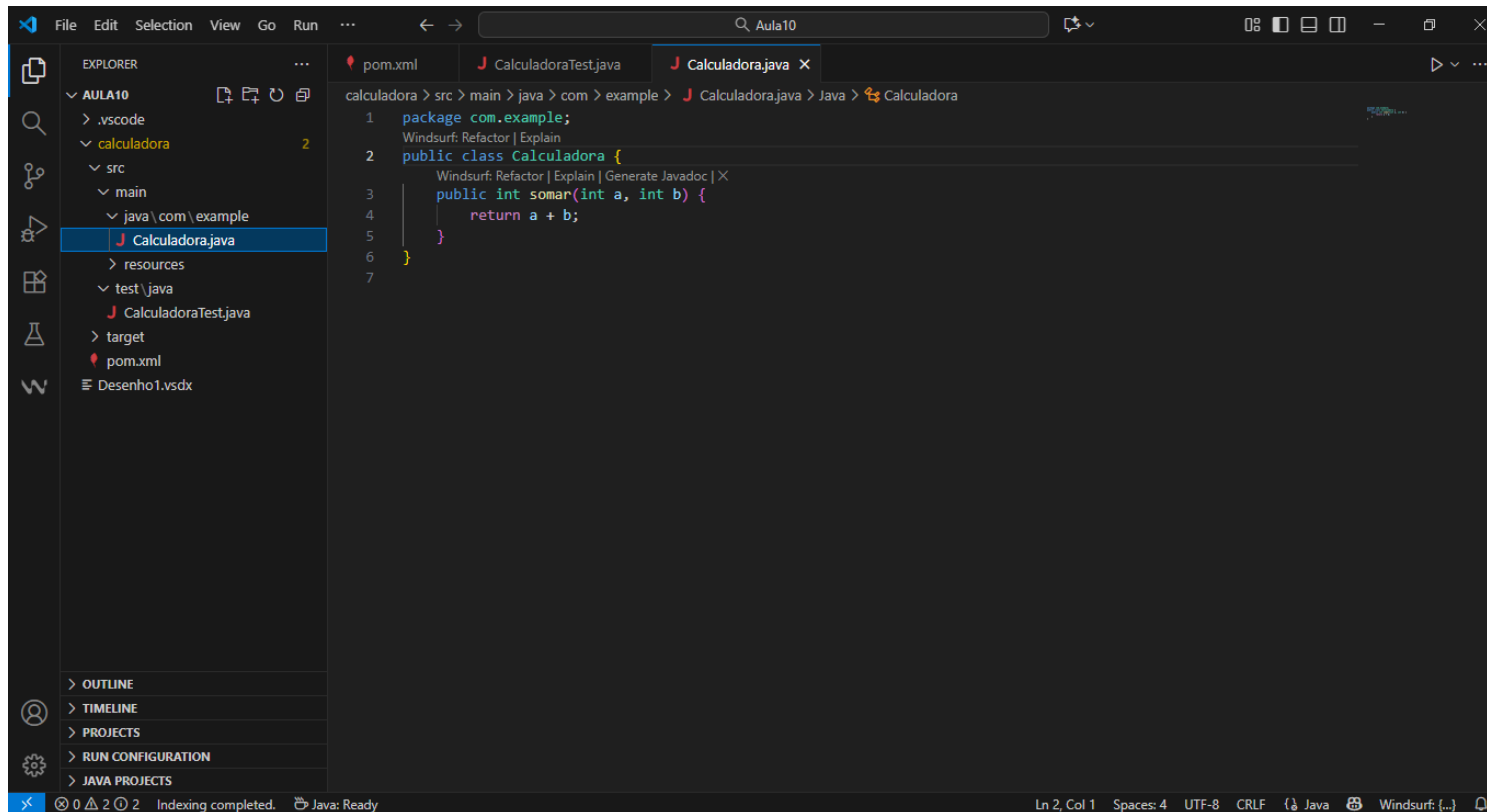
```
package com.exemplo;  
  
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *Calculadora.java*



Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest.java*

```
File Edit Selection View Go Run ...
pom.xml CalculadoraTest.java X Calculadora.java
calculadora > src > test > java > J CalculadoraTest.java > ...
1 import com.example.Calculadora;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 Windsurf: Refactor | Explain
class CalculadoraTest {
6
7   Windsurf: Refactor | Explain | Generate Javadoc | X
8   @Test
9   void testSomar() {
10     Calculadora calc = new Calculadora();
11     assertEquals(expected:5, calc.somar(a:2, b:3));
12   }
13
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS Filter (e.g. text, !exclude, \.esca... Java Single Test Run

```
-----
Running CalculadoraTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in CalculadoraTest

Results:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

-----
BUILD SUCCESS
-----
Total time: 2.006 s
Finished at: 2025-10-11T23:05:00-03:00
-----
Please start a debug session to evaluate expressions
```

Ln 13, Col 1 Spaces: 4 UTF-8 CRLF Java Windsurf: {...}

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Fácil: Classe *CalculadoraTest.java*

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

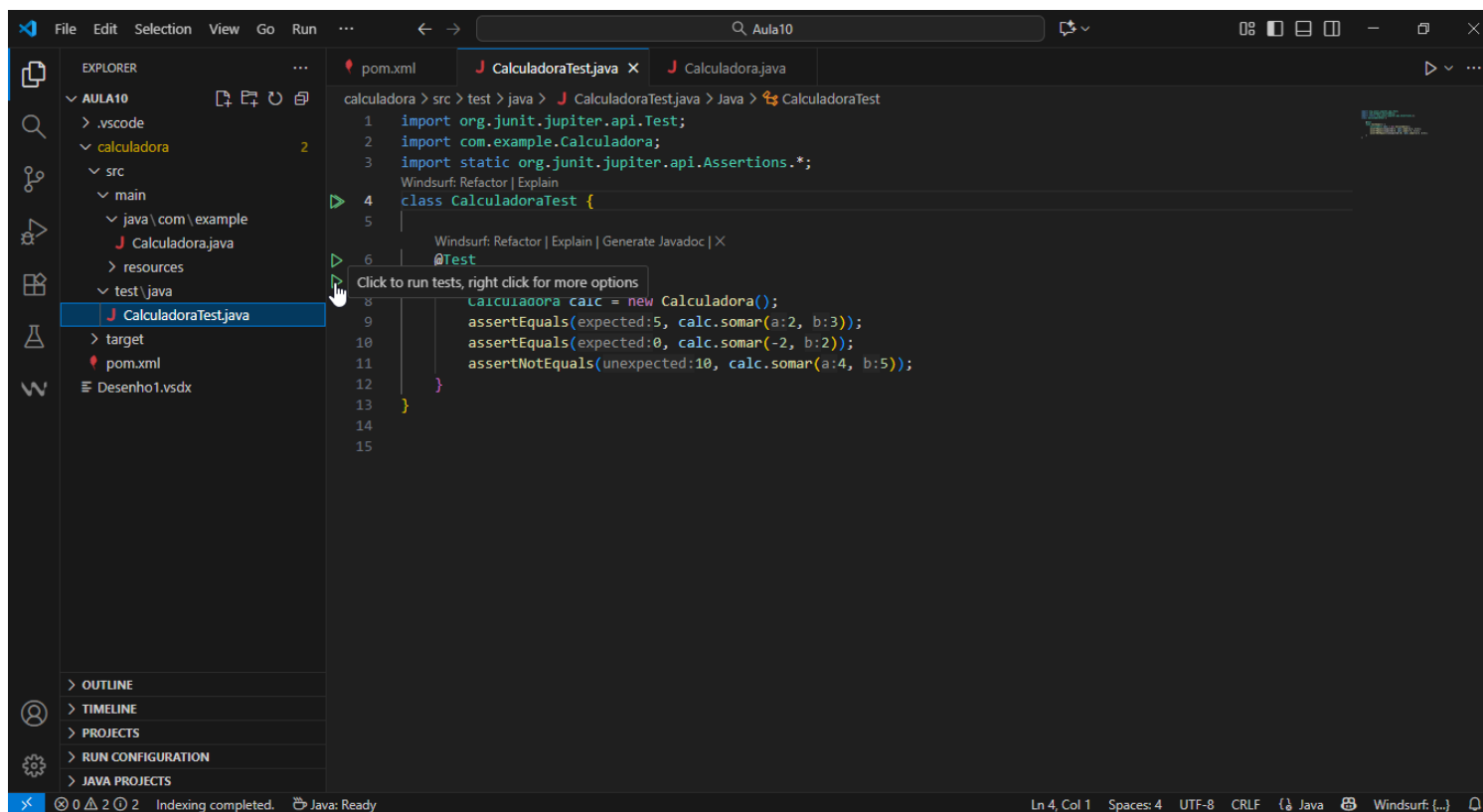
    @Test
    void testSomar() {
        Calculadora calc = new Calculadora();
        assertEquals(5, calc.somar(2, 3));
        assertEquals(0, calc.somar(-2, 2));
        assertNotEquals(10, calc.somar(4, 5));
    }
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

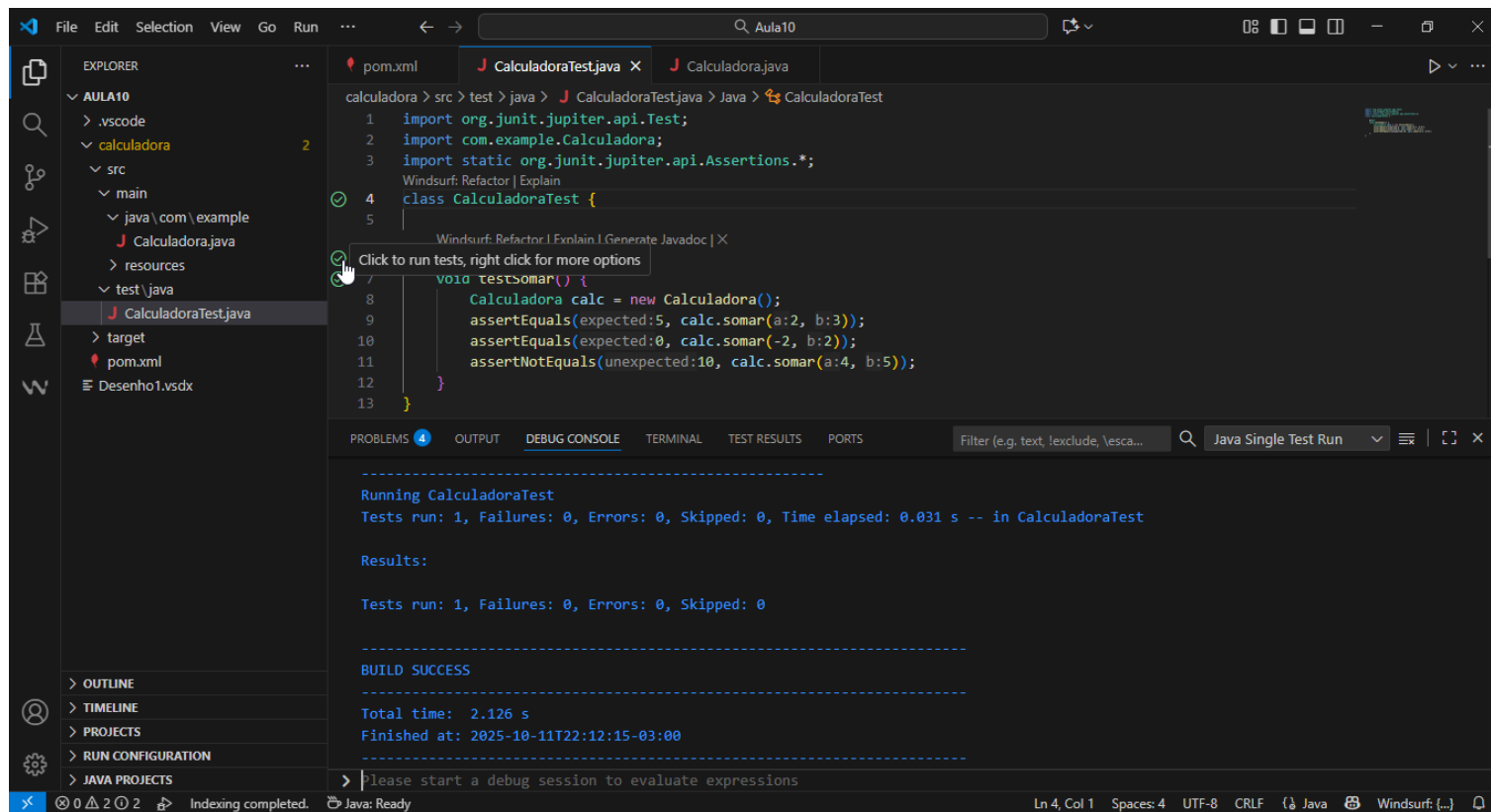
- Executa o teste unitário do método *somar()*:



Como são?

» Testes Unitários:

- Resultado do teste unitário do método `somar( )`:



```
File Edit Selection View Go Run ... Aula10
pom.xml CalculadoraTest.java Calculadora.java
calculadora > src > test > java > J CalculadoraTest.java > Java > CalculadoraTest
1 import org.junit.jupiter.api.Test;
2 import com.example.Calculadora;
3 import static org.junit.jupiter.api.Assertions.*;
4 class CalculadoraTest {
5
6     Click to run tests, right click for more options
7     void testSomar() {
8         Calculadora calc = new Calculadora();
9         assertEquals(expected:5, calc.somar(a:2, b:3));
10        assertEquals(expected:0, calc.somar(-2, b:2));
11        assertEquals(unexpected:10, calc.somar(a:4, b:5));
12    }
13 }

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
Filter (e.g. text, \exclude, \esca... Java Single Test Run
Running CalculadoraTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.031 s -- in CalculadoraTest

Results:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Total time: 2.126 s
Finished at: 2025-10-11T22:12:15-03:00

Please start a debug session to evaluate expressions
Ln 4, Col 1 Spaces: 4 UTF-8 CRLF Java Windsurf: {...}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Médio: Classe *ValidadorSenha*

ValidadorSenha

- validar(String): boolean

Como são?

» Testes Unitários:

- Nível Médio: Classe *ValidadorSenhaTest*
 - Lógica condicional múltipla e expressões regulares;
 - Uso de *assertTrue* e *assertFalse*.

ValidadorSenhaTest

- testValidarSenha(): void

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Médio: Classe *ValidadorSenhaTest.java*

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ValidadorSenhaTest {

    @Test
    void testValidarSenha() {
        ValidadorSenha val = new ValidadorSenha();
        assertTrue(val.validar("Senha123"));
        assertFalse(val.validar("senha123")); // falta maiúscula
        assertFalse(val.validar("SENHA"));    // falta número
        assertFalse(val.validar("123456"));    // falta letra
        assertFalse(val.validar(""));          // vazia
    }
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Médio: Classe *ValidadorSenha.java*

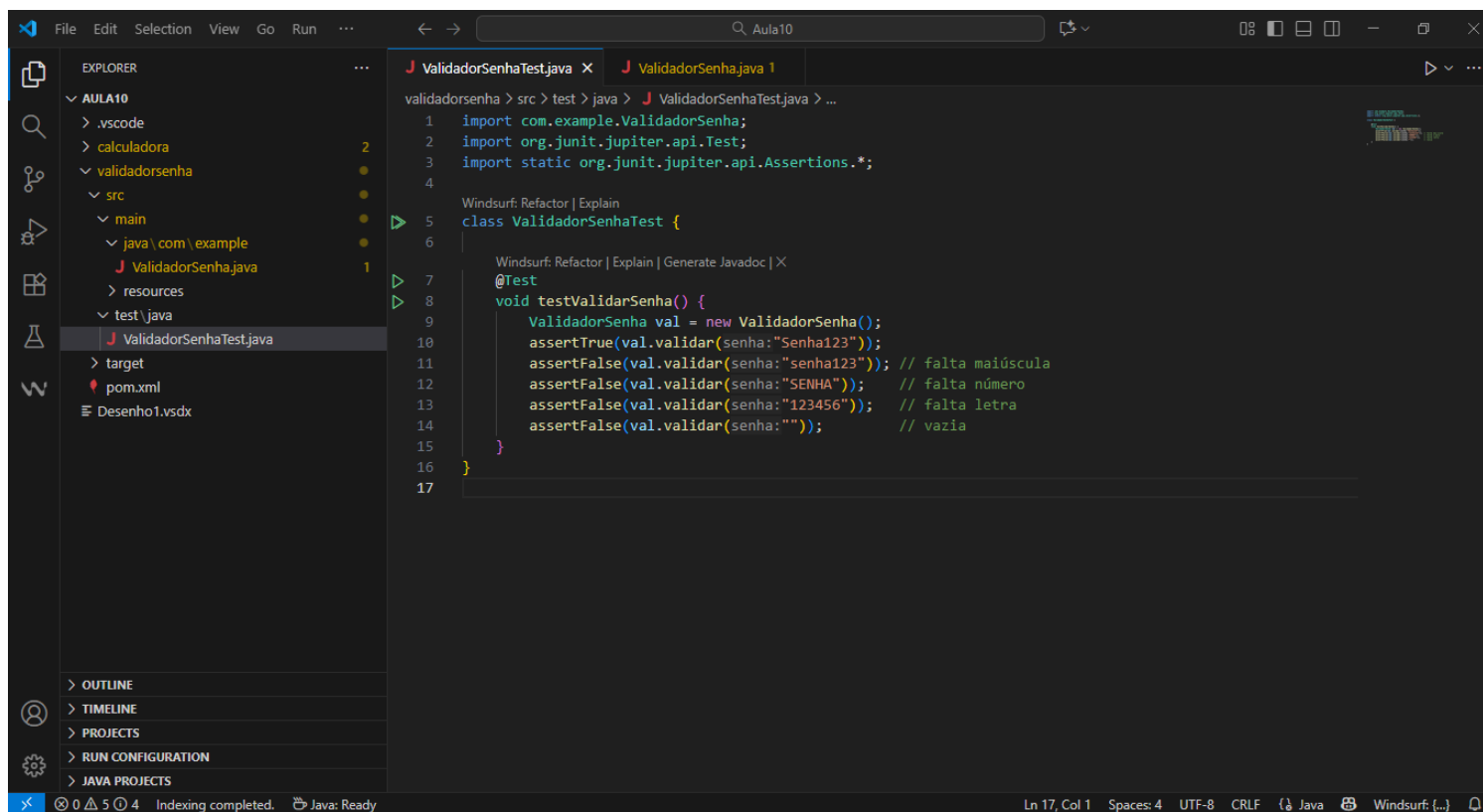
```
public class ValidadorSenha {  
    public boolean validar(String senha) {  
        if (senha == null) return false;  
        if (senha.length() < 8) return false;  
        if (!senha.matches(".*[A-Z].*")) return false;  
        if (!senha.matches(".*[0-9].*")) return false;  
        return true;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Executa o teste unitário do método *validar()*:



The screenshot shows a code editor with a Java unit test file named `ValidadorSenhaTest.java`. The test class uses JUnit 4 annotations and assertions to verify the behavior of the `validar()` method in the `ValidadorSenha` class. The test method `testValidarSenha()` creates an instance of `ValidadorSenha` and performs several assertions: `assertTrue` for a valid password, `assertFalse` for passwords missing a lowercase letter, a number, or a letter, and `assertFalse` for an empty password. The IDE interface includes a file explorer on the left showing the project structure, a top toolbar with standard editing and running actions, and a status bar at the bottom indicating the current line and column.

```
1 import com.example.ValidadorSenha;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class ValidadorSenhaTest {
6
7     @Test
8     void testValidarSenha() {
9         ValidadorSenha val = new ValidadorSenha();
10        assertTrue(val.validar(senha:"Senha123"));
11        assertFalse(val.validar(senha:"senha123")); // falta maiúscula
12        assertFalse(val.validar(senha:"SENHA")); // falta número
13        assertFalse(val.validar(senha:"123456")); // falta letra
14        assertFalse(val.validar(senha:"")); // vazia
15    }
16 }
17
```

Como são?

» Testes Unitários:

- Resultado do teste unitário do método *validar()*:

The screenshot displays the Visual Studio Code interface. On the left, the Explorer sidebar shows a project structure with folders like 'aula10', 'calculadora', and 'validadorsenha'. The file 'ValidadorSenhaTest.java' is selected. The main editor shows the code for 'ValidadorSenhaTest.java', which includes imports for 'com.example.ValidadorSenha', 'org.junit.jupiter.api.Test', and 'org.junit.jupiter.api.Assertions'. The test class 'ValidadorSenhaTest' contains a single test method 'testValidarSenha()' that uses 'assertTrue', 'assertFalse', and 'assertFalse' to validate password rules. Below the code editor, the 'DEBUG CONSOLE' tab is active, showing the output of running the test. The output indicates that all tests passed successfully, with a total time of 1.481 seconds and a completion time of 2025-10-11T23:41:36-03:00.

```
validadorsenha > src > test > java > J ValidadorSenhaTest.java > ...
1  import com.example.ValidadorSenha;
2  import org.junit.jupiter.api.Test;
3  import static org.junit.jupiter.api.Assertions.*;
4
5  Windsurf: Refactor | Explain
class ValidadorSenhaTest {
6
7  Windsurf: Refactor | Explain | Generate Javadoc | X
8  @Test
9  void testValidarSenha() {
10     ValidadorSenha val = new ValidadorSenha();
11     assertTrue(val.validar(senha:"Senha123"));
12     assertFalse(val.validar(senha:"senha123")); // falta maiúscula
13     assertFalse(val.validar(senha:"SENHA")); // falta número
14     assertFalse(val.validar(senha:"123456")); // falta letra
15 }
```

Running ValidadorSenhaTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.037 s -- in ValidadorSenhaTest

Results:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Total time: 1.481 s
Finished at: 2025-10-11T23:41:36-03:00

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Nível Complexo: Classe *ConversorMoedas*

ConversorMoedas

-
- ConversorMoedas()
 - converter(String, String, double): double

Como são?

» Testes Unitários:

- Nível Complexo: Classe *ConversorMoedasTest*
 - Exceção (*assertThrows*) e Estruturas de dados (Map);
 - Cenários de erro e sucesso.

ConversorMoedasTest

-
- testConversaoValida(): void
 - testConversaoNaoSuportada(): void
 - testValorNegativo(): void

Como são?

» Testes Unitários:

- Nível Complexo: Classe *ConversorMoedasTest.java* (1/2)

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ConversorMoedasTest {

    @Test
    void testConversaoValida() {
        ConversorMoedas conv = new ConversorMoedas();
        assertEquals(50.0, conv.converter("USD", "BRL", 10.0));
    }
}
```

Como são?

» Testes Unitários:

- Nível Complexo: Classe *ConversorMoedasTest.java* (2/2)

```
@Test
void testConversaoNaoSuportada() {
    ConversorMoedas conv = new ConversorMoedas();
    assertThrows(IllegalArgumentException.class, () -> {
        conv.converter("JPY", "BRL", 100);
    });
}

@Test
void testValorNegativo() {
    ConversorMoedas conv = new ConversorMoedas();
    assertThrows(IllegalArgumentException.class, () -> {
        conv.converter("USD", "BRL", -10);
    });
}
```

Como são?

» Testes Unitários:

- Nível Complexo: Classe *ConversorMoedas.java*

```
import java.util.HashMap;
import java.util.Map;

public class ConversorMoedas {
    private final Map<String, Double> taxas = new HashMap<>();

    public ConversorMoedas() {
        taxas.put("USD_BRL", 5.0);
        taxas.put("EUR_BRL", 6.0);
    }

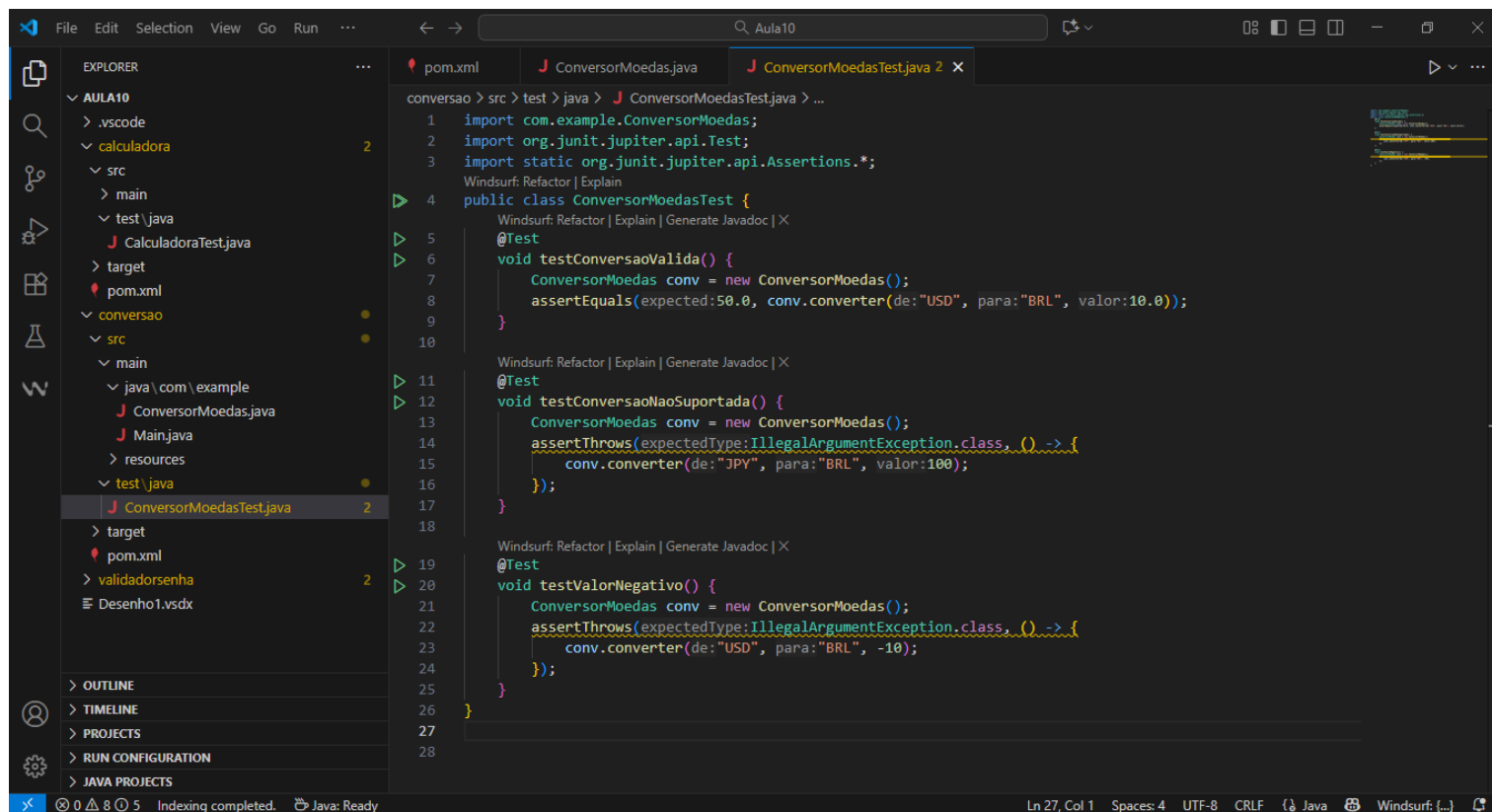
    public double converter(String de, String para, double valor) {
        if (valor < 0) throw new IllegalArgumentException("Valor não pode ser negativo");
        String chave = de + "_" + para;
        if (!taxas.containsKey(chave))
            throw new IllegalArgumentException("Conversão não suportada");
        return valor * taxas.get(chave);
    }
}
```

Aula 10 – Teoria

Como são?

» Testes Unitários:

- Executa o teste unitário dos métodos de conversão:



The screenshot shows a code editor with three tabs: `pom.xml`, `ConversorMoedas.java`, and `ConversorMoedasTest.java 2`. The `ConversorMoedasTest.java` file contains the following code:

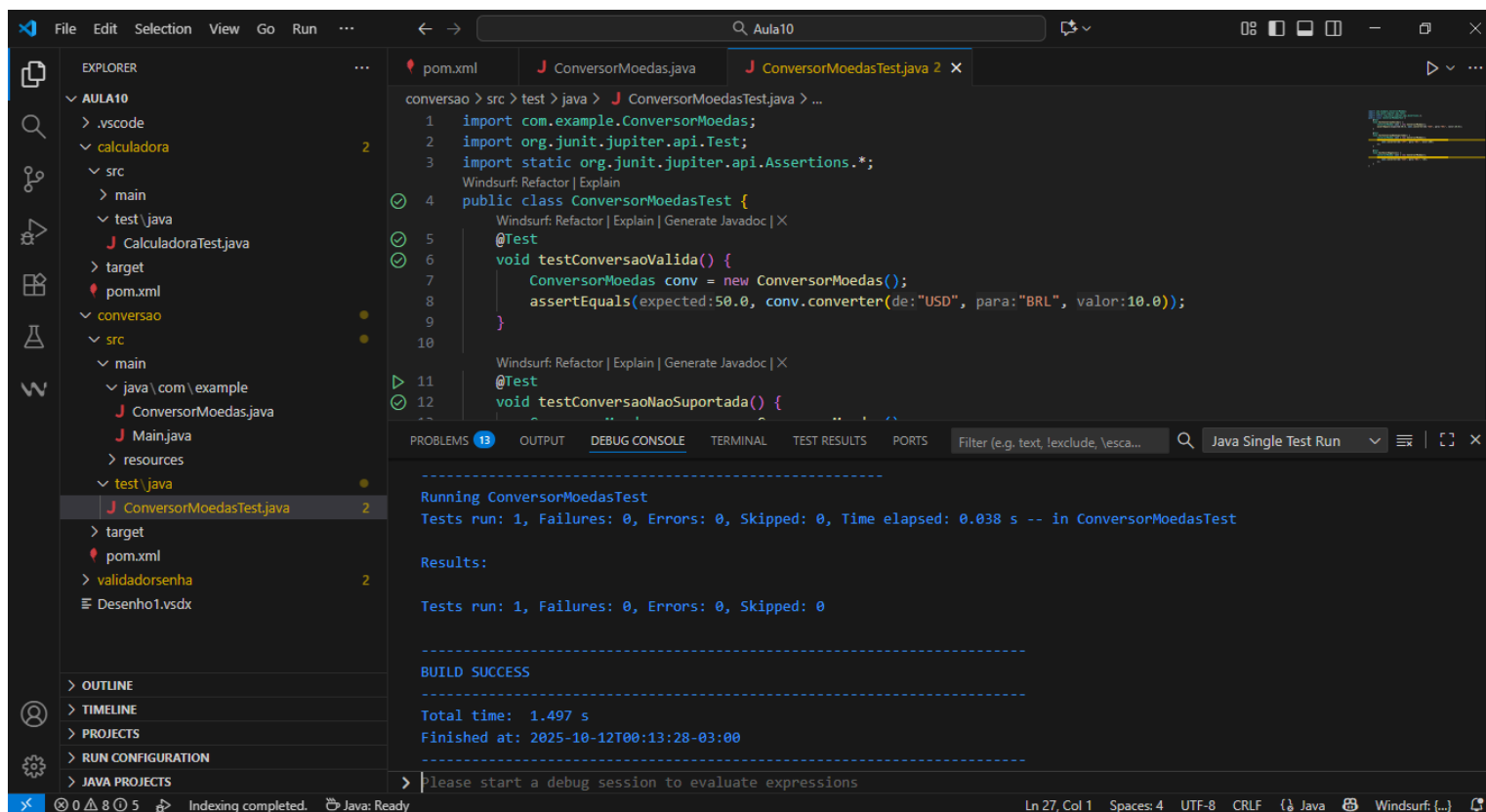
```
1 import com.example.ConversorMoedas;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4 public class ConversorMoedasTest {
5     @Test
6     void testConversaoValida() {
7         ConversorMoedas conv = new ConversorMoedas();
8         assertEquals(expected:50.0, conv.converter(de:"USD", para:"BRL", valor:10.0));
9     }
10
11     @Test
12     void testConversaoNaoSuportada() {
13         ConversorMoedas conv = new ConversorMoedas();
14         assertThrows(expectedType:IllegalArgumentException.class, () -> {
15             conv.converter(de:"JPY", para:"BRL", valor:100);
16         });
17     }
18
19     @Test
20     void testValorNegativo() {
21         ConversorMoedas conv = new ConversorMoedas();
22         assertThrows(expectedType:IllegalArgumentException.class, () -> {
23             conv.converter(de:"USD", para:"BRL", -10);
24         });
25     }
26 }
27
28
```

The IDE interface includes a sidebar with a file explorer showing the project structure, including folders like `.vscode`, `calculadora`, `src`, `main`, `test\java`, `target`, `conversao`, and `src`. The status bar at the bottom indicates "Ln 27, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Java".

Como são?

» Testes Unitários:

- Resultado do teste unitário dos métodos de conversão:



The screenshot shows an IDE with the following components:

- EXPLORER:** A tree view on the left showing the project structure. The 'test' directory is expanded, showing 'ConverterMoedasTest.java'.
- EDITOR:** The main window displays the source code of 'ConverterMoedasTest.java'. It includes imports for 'com.example.ConversorMoedas', 'org.junit.jupiter.api.Test', and 'org.junit.jupiter.api.Assertions'. The code defines two test methods: 'testConversaoValida()' and 'testConversaoNaoSuportada()'.
- PROBLEMS:** A panel at the bottom shows the results of the test run. It indicates that all tests passed successfully.

```
1 import com.example.ConversorMoedas;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4 public class ConverterMoedasTest {
5     @Test
6     void testConversaoValida() {
7         ConverterMoedas conv = new ConverterMoedas();
8         assertEquals(expected:50.0, conv.converter(de:"USD", para:"BRL", valor:10.0));
9     }
10
11     @Test
12     void testConversaoNaoSuportada() {
```

Running ConverterMoedasTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.038 s -- in ConverterMoedasTest

Results:
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

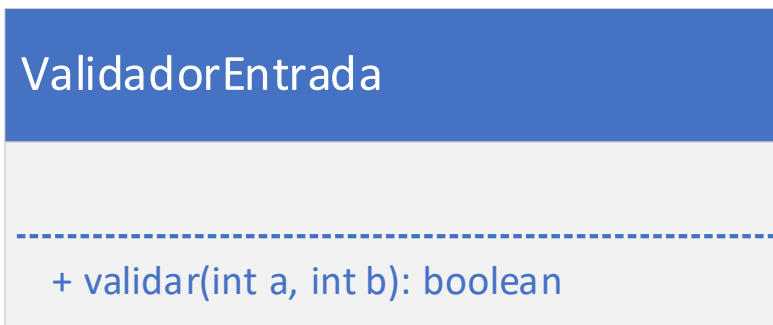
Total time: 1.497 s
Finished at: 2025-10-12T00:13:28-03:00

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Fácil: Classe *ValidadorEntrada*



Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Fácil: Classe *CalculadoraSegura*

CalculadoraSegura

+ somarSeguramente(int, int): Integer

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Fácil: Classe *Calculadora*

Calculadora

- somar(int, int): int

Como são?

» Testes de Integração:

- Nível Fácil: Classe *CalculadoraSeguraIT*
 - Integração de duas classes simples;
 - Retorno condicional em caso de falha de validação.

CalculadoraSeguraIT

+ testSomaComValidacao(): void

Como são?

» Testes de Integração:

- Nível Fácil: Classe *CalculadoraSeguraIT.java*

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraSeguraIT {

    @Test
    void testSomaComValidacao() {
        CalculadoraSegura cs = new CalculadoraSegura();
        assertEquals(7, cs.somarSeguramente(3, 4));
        assertNull(cs.somarSeguramente(-1, 5));
    }
}
```

Como são?

» Testes de Integração:

- Nível Fácil: Classe *ValidadorEntrada.java*

```
public class ValidadorEntrada {  
    public boolean validar(int a, int b) {  
        return a >= 0 && b >= 0;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Fácil: Classe *CalculadoraSegura.java*

```
public class CalculadoraSegura {  
    private final ValidadorEntrada validador = new ValidadorEntrada();  
    private final Calculadora calc = new Calculadora();  
  
    public Integer somarSeguramente(int a, int b) {  
        if (!validador.validar(a, b)) return null;  
        return calc.somar(a, b);  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Fácil: Classe *Calculadora.java*

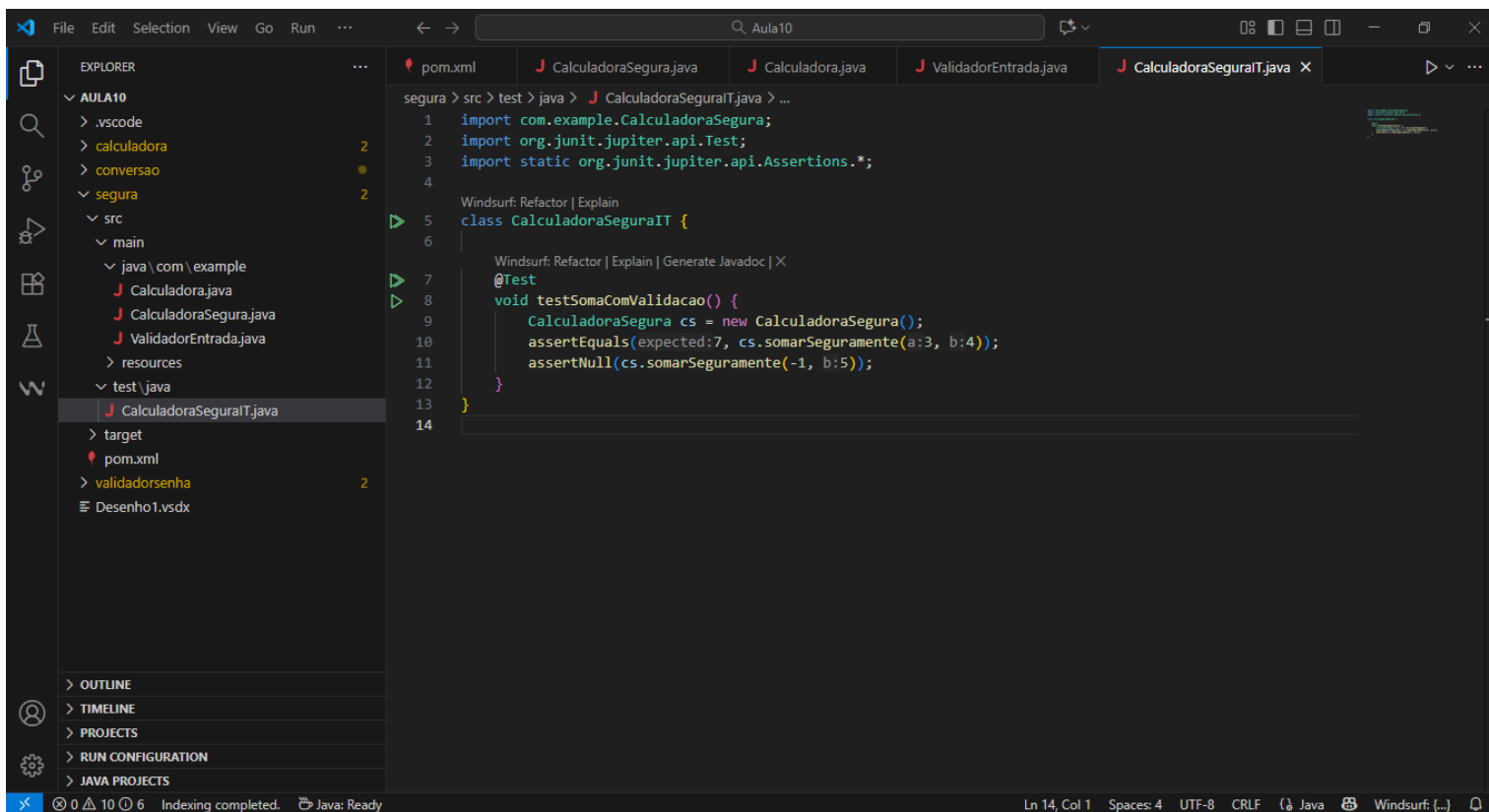
```
package com.exemplo;  
  
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Executa o teste de integração:



The screenshot shows a code editor with the following content:

```
1 import com.example.CalculadoraSegura;  
2 import org.junit.jupiter.api.Test;  
3 import static org.junit.jupiter.api.Assertions.*;  
4  
5 class CalculadoraSeguraIT {  
6  
7     @Test  
8     void testSomaComValidacao() {  
9         CalculadoraSegura cs = new CalculadoraSegura();  
10        assertEquals(expected:7, cs.somarSeguramente(a:3, b:4));  
11        assertNull(cs.somarSeguramente(-1, b:5));  
12    }  
13 }  
14
```

The IDE interface includes a sidebar with a file explorer showing a project structure with folders like 'aula10', 'calculadora', 'conversao', 'segura', and 'test'. The main editor area displays the code for 'CalculadoraSeguraIT.java'. The status bar at the bottom indicates 'Ln 14, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'.

Como são?

» Testes de Integração:

- Resultado do teste de integração:

The screenshot displays an IDE interface with the following components:

- EXPLORER:** Shows a project structure with folders like `.vscode`, `calculadora`, `conversao`, `segura`, `src`, `main`, `test`, `target`, `validadorsenha`, and `Desenho1.vsd`. The `CalculadoraSeguraIT.java` file is selected under `test/java`.
- Editor:** Displays the code for `CalculadoraSeguraIT.java`. The code includes imports for `com.example.CalculadoraSegura`, `org.junit.jupiter.api.Test`, and `org.junit.jupiter.api.Assertions.*`. It defines a class `CalculadoraSeguraIT` with a test method `testSomaComValidacao()` that uses `assertEquals` and `assertNull` to verify calculations.
- DEBUG CONSOLE:** Shows the output of the test run. It indicates that the test `Running CalculadoraSeguraIT` was successful, with 1 test run, 0 failures, 0 errors, and 0 skipped tests. The total time elapsed was 0.059 seconds.
- STATUS BAR:** At the bottom, it shows "Ln 14, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Java".

Como são?

» Testes de Integração:

- Nível Médio: Classe *RepositorioUsuarios*

RepositorioUsuarios

+ salvar(String, String): void
+ buscarSenha(String): String

Como são?

» Testes de Integração:

- Nível Médio: Classe *ServicoAutenticacao*

ServicoAutenticacao

+ ServicoAutenticacao(RepositorioUsuario)
+ autenticar(String, String): boolean

Como são?

» Testes de Integração:

- Nível Médio: Classe *ServicoAutenticacaoIT*
 - Interação entre camadas (repositório ↔ serviço).
 - Estado persistido em memória simulando BD.

ServicoAutenticacaoIT

+ testAutenticacaoCompleta(): void

Como são?

» Testes de Integração:

- Nível Médio: Classe *ServicoAutenticacaoIT.java*

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ServicoAutenticacaoIT {

    @Test
    void testAutenticacaoCompleta() {
        RepositorioUsuarios repo = new RepositorioUsuarios();
        repo.salvar("Alice", "1234");
        ServicoAutenticacao servico = new ServicoAutenticacao(repo);

        assertTrue(servico.autenticar("Alice", "1234"));
        assertFalse(servico.autenticar("Alice", "0000"));
        assertFalse(servico.autenticar("Bob", "1234"));
    }
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Nível Médio: Classe *RepositorioUsuarios.java*

```
import java.util.HashMap;  
import java.util.Map;  
  
public class RepositorioUsuarios {  
    private final Map<String, String> usuarios = new HashMap<>();  
  
    public void salvar(String nome, String senha) {  
        usuarios.put(nome, senha);  
    }  
  
    public String buscarSenha(String nome) {  
        return usuarios.get(nome);  
    }  
}
```

Como são?

» Testes de Integração:

- Nível Médio: Classe *ServicoAutenticacao.java*

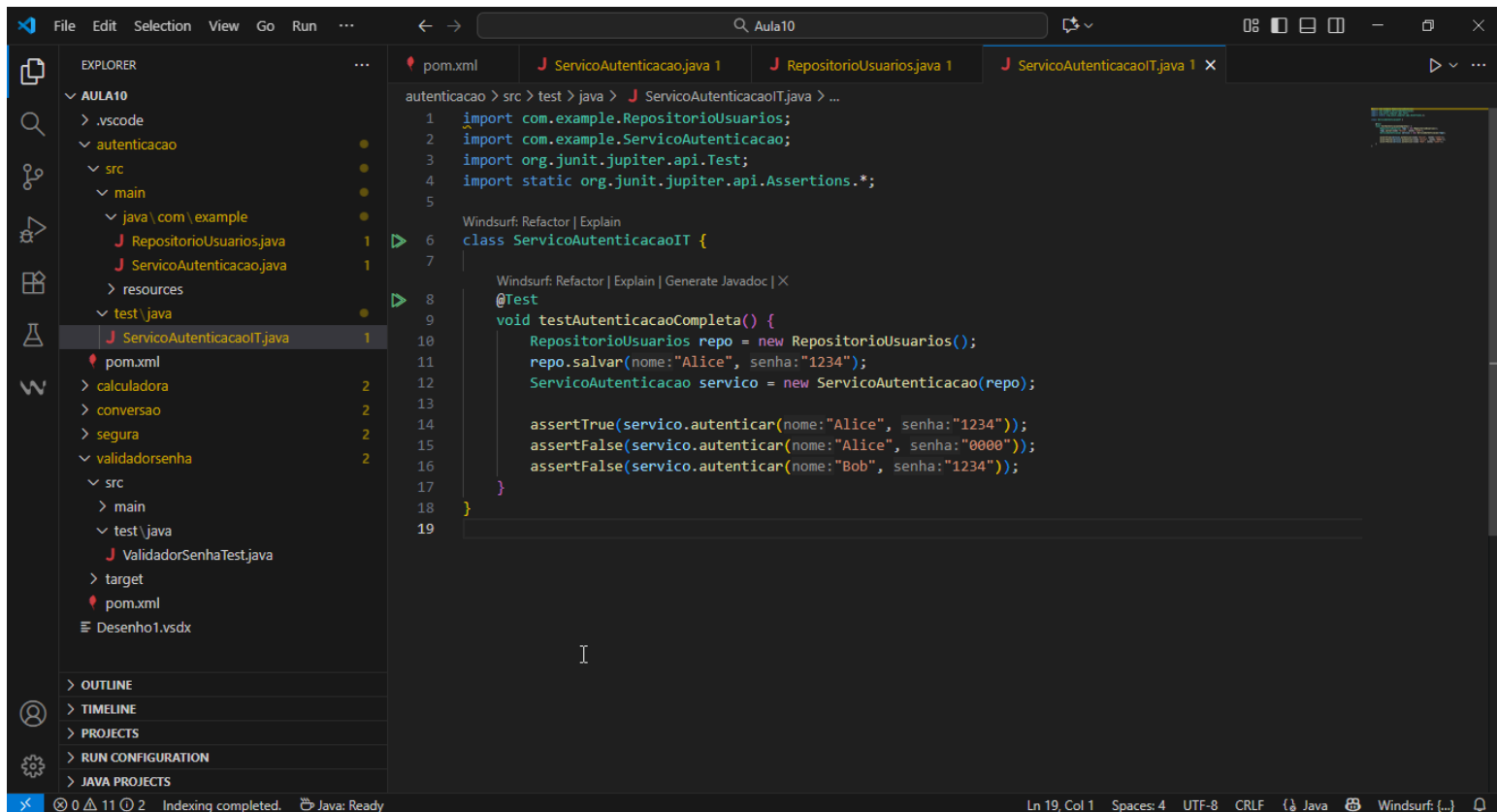
```
public class ServicoAutenticacao {  
    private final RepositorioUsuarios repo;  
  
    public ServicoAutenticacao(RepositorioUsuarios repo) {  
        this.repo = repo;  
    }  
  
    public boolean autenticar(String nome, String senha) {  
        String senhaSalva = repo.buscarSenha(nome);  
        return senhaSalva != null && senhaSalva.equals(senha);  
    }  
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Executa o teste de integração:



The screenshot shows an IDE with the following components:

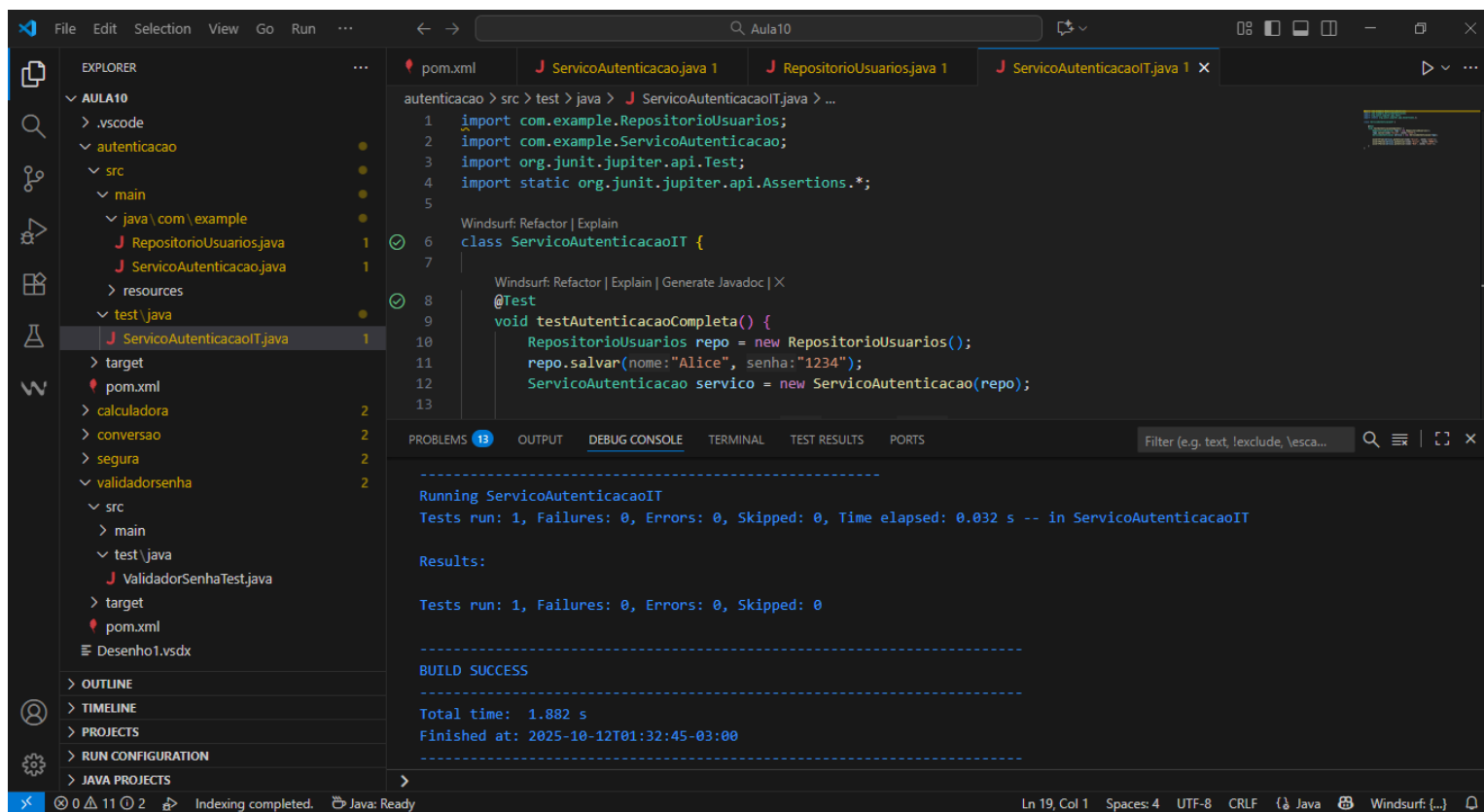
- EXPLORER:** A tree view on the left showing the project structure. The 'test' directory is expanded, showing 'ServicoAutenticacaoIT.java' selected.
- Code Editor:** The main area displays the content of 'ServicoAutenticacaoIT.java'. The code includes imports for 'com.example.RepositorioUsuarios', 'com.example.ServicoAutenticacao', 'org.junit.jupiter.api.Test', and 'org.junit.jupiter.api.Assertions'. It defines a class 'ServicoAutenticacaoIT' with a test method 'testAutenticacaoCompleta()' that uses 'assertTrue' and 'assertFalse' to verify the behavior of the 'ServicoAutenticacao' service.
- Terminal:** At the bottom, a terminal window shows the command 'autenticacao > src > test > java > J ServicoAutenticacaoIT.java > ...'.
- Status Bar:** The bottom status bar indicates 'Ln 19, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java: Ready'.

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Resultado do teste de integração:



The screenshot displays an IDE interface with the following components:

- EXPLORER:** Shows a project structure for 'AULA10' with folders like 'src', 'test', and 'target'. The file 'ServicoAutenticacaoIT.java' is selected in the 'test' folder.
- Editor:** Displays the code for 'ServicoAutenticacaoIT.java'. It includes imports for 'com.example.RepositorioUsuarios', 'com.example.ServicoAutenticacao', 'org.junit.jupiter.api.Test', and 'org.junit.jupiter.api.Assertions.*'. The code defines a 'ServicoAutenticacaoIT' class with a '@Test' method 'testAutenticacaoCompleta()' that creates a 'RepositorioUsuarios' instance, saves a user, and calls 'ServicoAutenticacao'.
- PROBLEMS:** Shows 13 problems, likely related to the test code.
- DEBUG CONSOLE:** Displays the output of the test execution. It shows 'Running ServicoAutenticacaoIT', 'Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.032 s -- in ServicoAutenticacaoIT', and 'Results: Tests run: 1, Failures: 0, Errors: 0, Skipped: 0'. It also shows 'BUILD SUCCESS' and 'Total time: 1.882 s'.

Como são?

» Testes de Integração:

- Nível Complexo: Classe *Pedido*

Pedido

```
- id: String  
- valor: double  
- pago: boolean  
-----  
+ Pedido(String, double)  
+ getId(): String  
+ getValor(): double  
+ isPago(): boolean  
+ marcarComoPago(): void
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ServicoPagamento*

ServicoPagamento

+ processarPagamento(Pedido, double): boolean

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidos*

ControladorPedidos

- + criarPedido(String, double): void
- + pagarPedido(String, double): boolean
- + getPedido(String): Pedido

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidosIT*
 - Integração de múltiplas classes;
 - Verificação de fluxo de negócio completo;
 - *Asserts* encadeados (*assertTrue*, *assertFalse*) validando estados antes e depois.

ControladorPedidosIT

+ testFluxoCompletoDePagamento(): void
+ testPagamentoInsuficiente(): void

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidosIT.java* (1/2)

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ControladorPedidosIT {

    @Test
    void testFluxoCompletoDePagamento() {
        ControladorPedidos ctrl = new ControladorPedidos();

        ctrl.criarPedido("P1", 100.0);
        assertFalse(ctrl.getPedido("P1").isPago());

        boolean sucesso = ctrl.pagarPedido("P1", 120.0);
        assertTrue(sucesso);
        assertTrue(ctrl.getPedido("P1").isPago());
    }
}
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidosIT.java* (2/2)

```
@Test
void testPagamentoInsuficiente() {
    ControladorPedidos ctrl = new ControladorPedidos();

    ctrl.criarPedido("P2", 200.0);
    boolean sucesso = ctrl.pagarPedido("P2", 150.0);
    assertFalse(sucesso);
    assertFalse(ctrl.getPedido("P2").isPago());
}
}
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *Pedido.java*

```
public class Pedido {  
    private String id;  
    private double valor;  
    private boolean pago;  
  
    public Pedido(String id, double valor) {  
        this.id = id;  
        this.valor = valor;  
        this.pago = false;  
    }  
  
    public String getId() { return id; }  
    public double getValor() { return valor; }  
    public boolean isPago() { return pago; }  
    public void marcarComoPago() { this.pago = true; }  
}
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ServicoPagamento.java*

```
public class ServicoPagamento {  
    public boolean processarPagamento(Pedido pedido, double valorPago) {  
        if (valorPago >= pedido.getValor()) {  
            pedido.marcarComoPago();  
            return true;  
        }  
        return false;  
    }  
}
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidos.java* (1/2)

```
public class ControladorPedidos {  
    private final Map<String, Pedido> pedidos = new HashMap<>();  
    private final ServicoPagamento servicoPagamento = new ServicoPagamento();  
  
    public void criarPedido(String id, double valor) {  
        pedidos.put(id, new Pedido(id, valor));  
    }  
}
```

Como são?

» Testes de Integração:

- Nível Complexo: Classe *ControladorPedidos.java* (2/2)

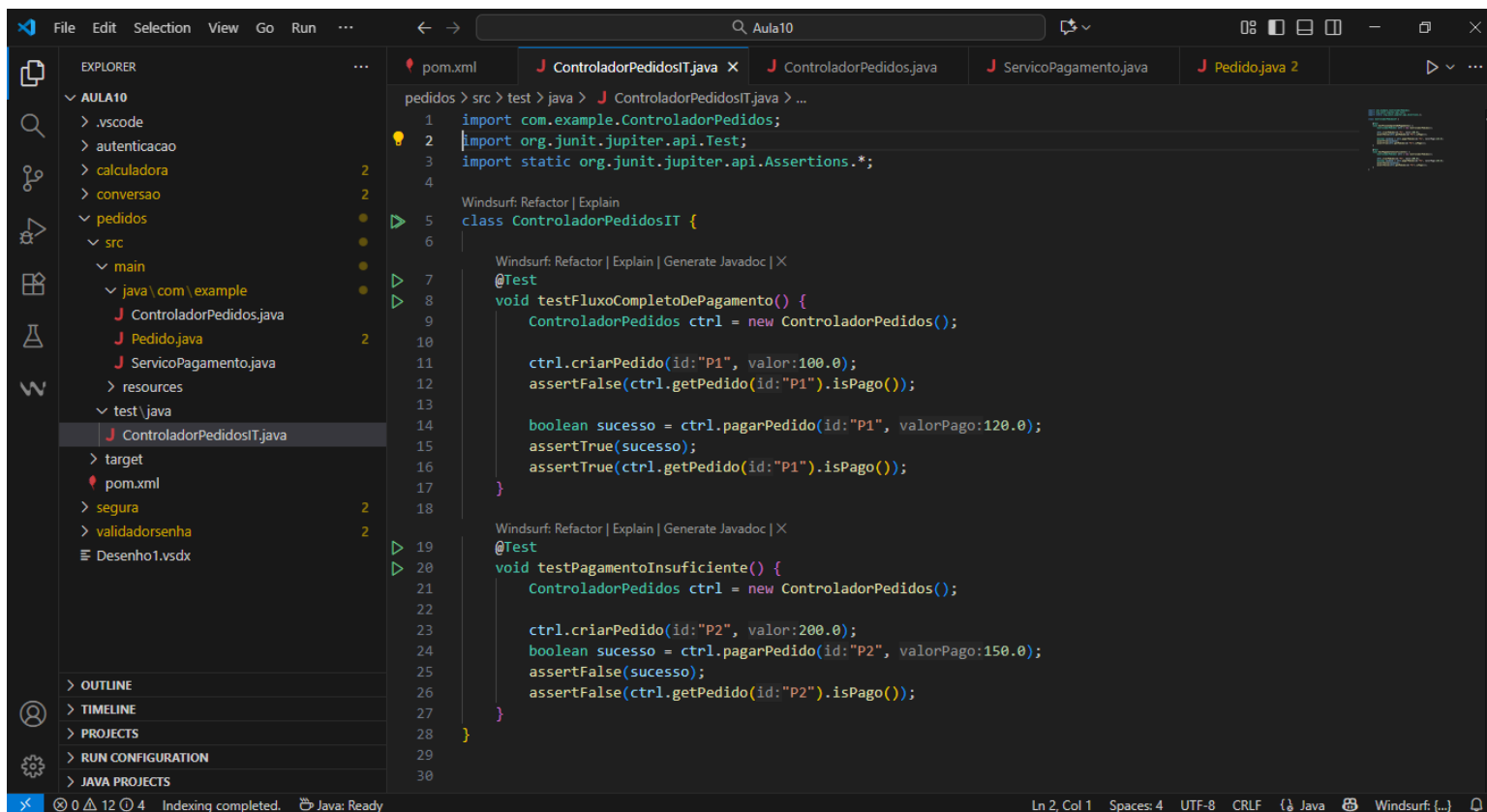
```
public boolean pagarPedido(String id, double valorPago) {  
    Pedido pedido = pedidos.get(id);  
    if (pedido == null) return false;  
    return servicoPagamento.processarPagamento(pedido, valorPago);  
}  
  
public Pedido getPedido(String id) {  
    return pedidos.get(id);  
}  
}
```

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Executa o teste de integração:



The screenshot shows an IDE with the following structure:

- EXPLORER:**
 - AULA10
 - .vscode
 - autenticacao
 - calculadora
 - conversao
 - pedidos
 - src
 - main
 - java\com\example
 - ControladorPedidos.java
 - Pedido.java
 - ServicoPagamento.java
 - resources
 - test\java
 - ControladorPedidosIT.java
 - target
 - pom.xml
 - segura
 - validadorsenha
 - Desenho1.vsd
 - OUTLINE
 - TIMELINE
 - PROJECTS
 - RUN CONFIGURATION
 - JAVA PROJECTS

- Code Editor:**
- File: pom.xml
- ControladorPedidosIT.java (active)
 - 1 import com.example.ControladorPedidos;
 - 2 import org.junit.jupiter.api.Test;
 - 3 import static org.junit.jupiter.api.Assertions.*;
 - 4
 - 5 class ControladorPedidosIT {
 - 6
 - 7 @Test
 - 8 void testFluxoCompletoDePagamento() {
 - 9 ControladorPedidos ctrl = new ControladorPedidos();
 - 10
 - 11 ctrl.criarPedido(id:"P1", valor:100.0);
 - 12 assertFalse(ctrl.getPedido(id:"P1").isPago());
 - 13
 - 14 boolean sucesso = ctrl.pagarPedido(id:"P1", valorPago:120.0);
 - 15 assertTrue(sucesso);
 - 16 assertTrue(ctrl.getPedido(id:"P1").isPago());
 - 17 }
 - 18
 - 19 @Test
 - 20 void testPagamentoInsuficiente() {
 - 21 ControladorPedidos ctrl = new ControladorPedidos();
 - 22
 - 23 ctrl.criarPedido(id:"P2", valor:200.0);
 - 24 boolean sucesso = ctrl.pagarPedido(id:"P2", valorPago:150.0);
 - 25 assertFalse(sucesso);
 - 26 assertFalse(ctrl.getPedido(id:"P2").isPago());
 - 27 }
 - 28 }
 - 29 }
 - 30 }
- ControladorPedidos.java
- ServicoPagamento.java
- Pedido.java 2

Windsurf: Refactor | Explain | Generate Javadoc | X

Windsurf: Refactor | Explain | Generate Javadoc | X

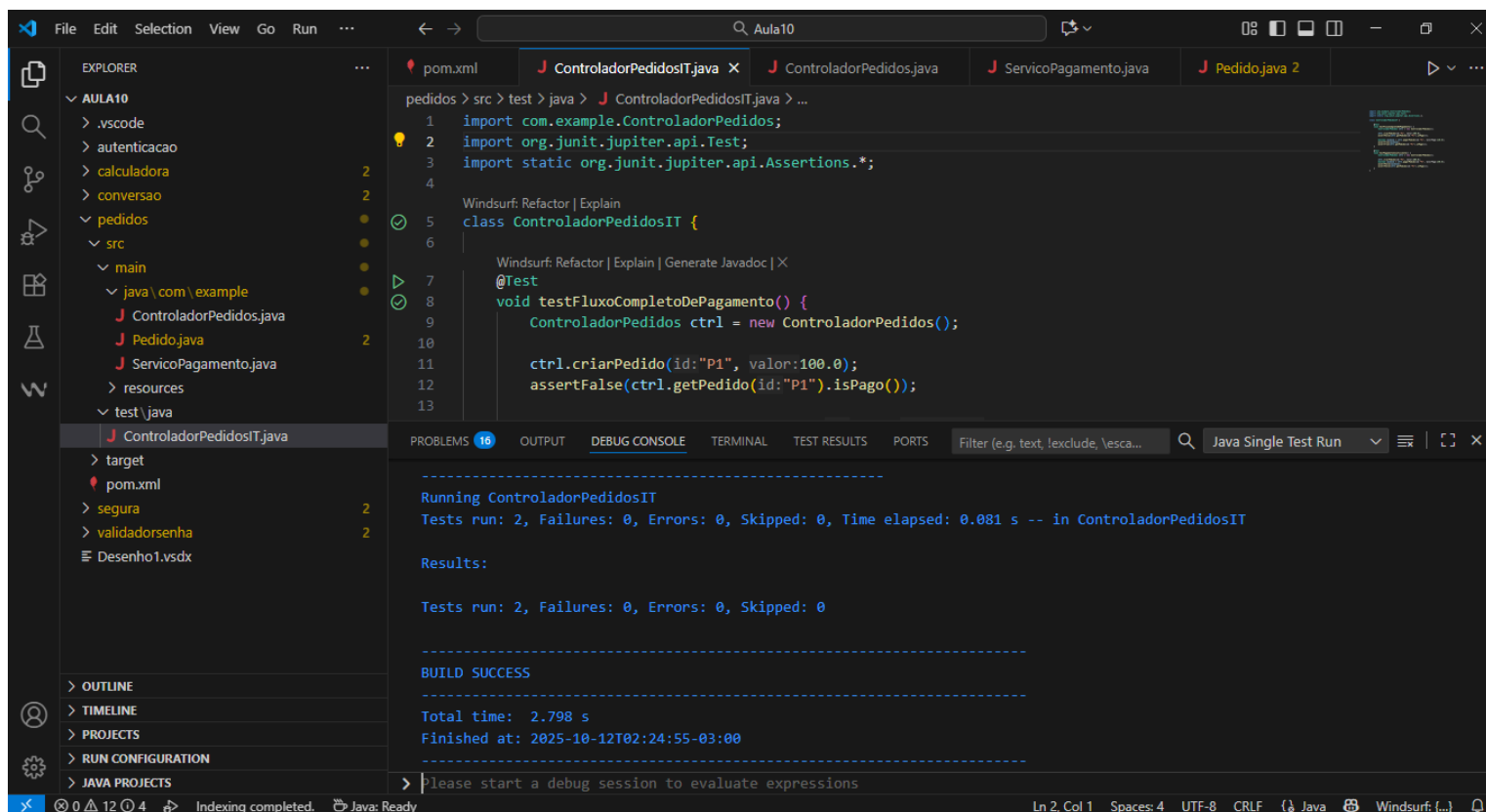
Ln 2, Col 1 Spaces: 4 UTF-8 CRLF Java Windsurf: [..]

Aula 10 – Teoria

Como são?

» Testes de Integração:

- Resultado do teste de integração:



```
File Edit Selection View Go Run ... Aula10
EXPLORER
  AULA10
    .vscode
    autenticao
    calculadora
    conversao
    pedidos
      src
        main
          java\com\example
            ControladorPedidos.java
            Pedido.java
            ServicoPagamento.java
          resources
          test\java
            ControladorPedidosIT.java
          target
            pom.xml
            segura
            validadorsenha
            Desenho1.vsd
  OUTLINE
  TIMELINE
  PROJECTS
  RUN CONFIGURATION
  JAVA PROJECTS

pom.xml ControladorPedidosIT.java ControladorPedidos.java ServicoPagamento.java Pedido.java 2
pedidos > src > test > java > J ControladorPedidosIT.java > ...
1 import com.example.ControladorPedidos;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class ControladorPedidosIT {
6
7   @Test
8   void testFluxoCompletoDePagamento() {
9     ControladorPedidos ctrl = new ControladorPedidos();
10
11     ctrl.criarPedido(id:"P1", valor:100.0);
12     assertFalse(ctrl.getPedido(id:"P1").isPago());
13
14   }
15 }

Windsurf: Refactor | Explain
Windsurf: Refactor | Explain | Generate Javadoc | X

Running ControladorPedidosIT
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.081 s -- in ControladorPedidosIT

Results:

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Total time: 2.798 s
Finished at: 2025-10-12T02:24:55-03:00

Please start a debug session to evaluate expressions
Ln 2, Col 1 Spaces: 4 UTF-8 CRLF Java Windsurf: [...]
```

Conclusões!

» **Automatização de Testes de *Software* com TDD:**

- É uma prática poderosa que promove a qualidade do *software* e a confiabilidade do código;
- Encoraja a escrita de testes automatizados desde o início do processo de desenvolvimento, o que ajuda a identificar problemas precocemente e a facilitar a manutenção do código ao longo do tempo;

Conclusões!

» **Automatização de Testes de *Software* com TDD:**

- É um processo que muda a maneira como os desenvolvedores executam a sua rotina de trabalho, pois vão se preocupar, primeiramente, em escrever um teste automatizado para algo que ainda não foi implementado e, somente depois disso, fazer sua implementação;
- A refatoração garante que, antes de se criar um novo teste, o código que acabou de ser criado passará por uma análise, visando retirar duplicações, renomear variáveis e deixar o código o mais limpo possível.

Aula 10 – Teoria

FIM